Новосибирск, Институт автоматики и электрометрии СО РАН

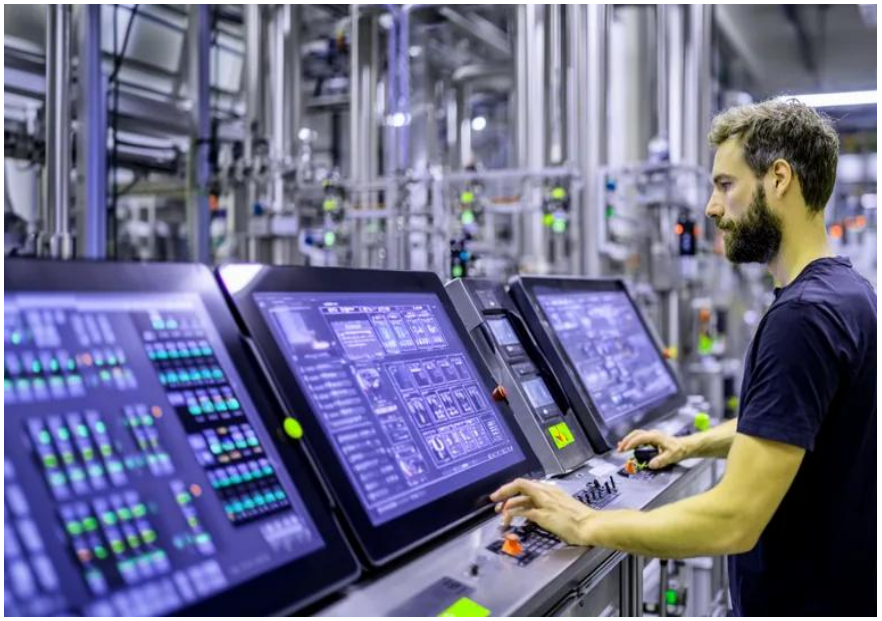
Процессно-ориентированное программирование систем промышленной автоматизации и перспективы использования ИИ при создании киберфизических систем

Лаборатория киберфизических систем
ИАиЭ СО РАН
Владимир Е. Зюбин

15.10.2026

Мотивация

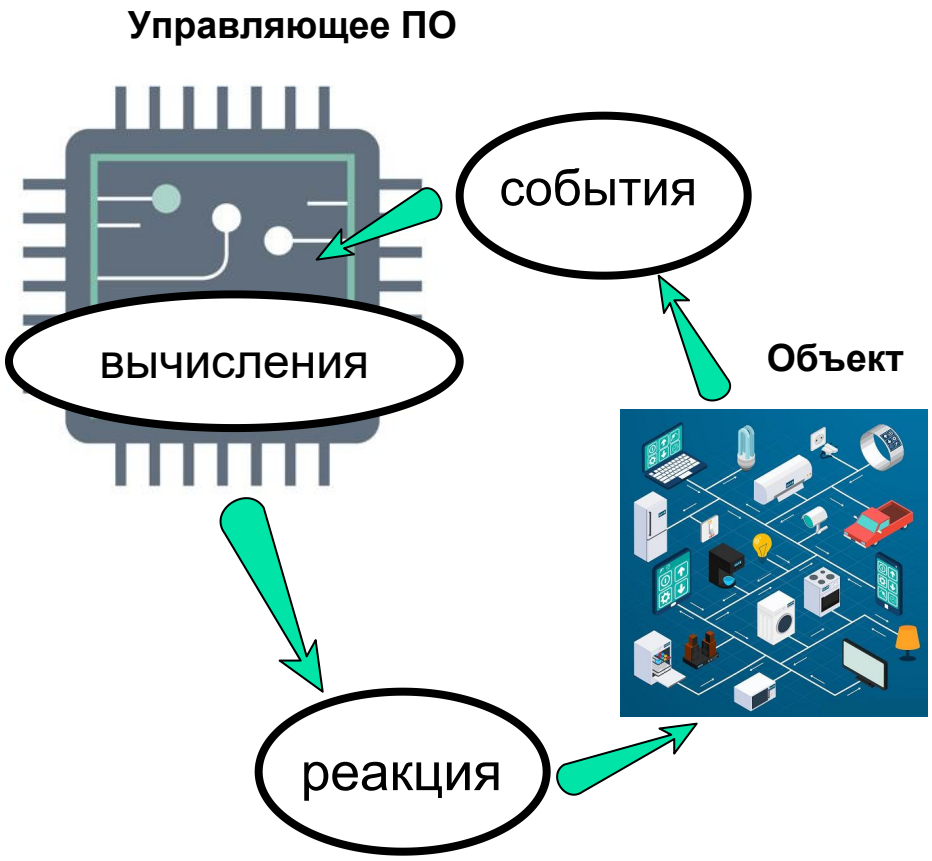
Разработка управляющего ПО – наиболее трудоемкая и дорогая часть общих затрат при создании промышленных киберфизических систем, или КФС (автоматизация технологических процессов, приборостроение, промышленный Интернет вещей, Индустрия 4.0). Растет сложность ПО систем управления, растут требования к надежности.



Текущая практика разработки – это либо языки IEC 61131-3/61499, либо еще более низкоуровневое программирование (C, C++, и др). Последствия – повышенные требования к разработчикам, вопросы качества ПО, высокая стоимость сопровождения.

Таким образом, разработка технологий создания ПО для современных промышленных систем автоматизации – перспективное направление исследований

Специфика промышленных КФС



- внешняя среда (открытость)
- неопределенное время функционирования
- событийная природа
- активная работа с временными интервалами (синхронизм)
- параллелизм
- неявное наличие алгоритма управления

Проблема программирования промышленных КФС

Программирование КФС задача комплексная, включает вопросы:

(а) теоретических моделей,

(б) инженерии требований,

(в) языков спецификации ПО,

(г) методов верификации ПО,

(д) средств анализа кода,

(е) инструментальных средств и их объединения в интегрированной среде разработки

Поэтому решение проблемы предполагает наличие стека высокоуровневых технологий.

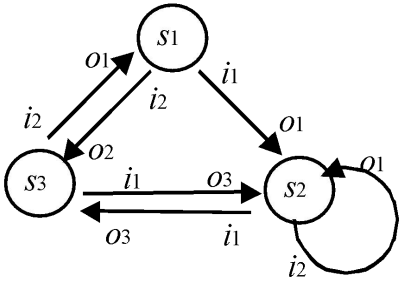
Классическая модель конечного автомата

Математическая модель КА

A = (***S***, ***I***, ***O***, ***Fs***, ***Fo***, s_1), где
S – множество состояний;
I – множество входных символов;
O – множество выходных символов;
Fs – функция переходов;
Fo – функция выходов;
s_1 – начальное состояние автомата.

Табличный и графический способы задания КА

Буквы входного алфавита	Состояния		
	s_1	s_2	s_3
i_1 i_2	s_2/o_1 s_3/o_2	s_3/o_3 s_2/o_1	s_2/o_3 s_1/o_1



Достоинства

Естественная открытость
Продолжительное функционирования
Событийность

Недостатки

- Отсутствие средств синхронизации
- Проблема блочного синтеза
- Отсутствие концепции переменной, арифметических операций, условных выражений
- Отсутствие механизмов структуризации и абстрагирования
- Ориентация на схемную реализацию и устаревшая терминология

Модернизированная модель конечного автомата

Конечный автомат – это циклически активизируемая событийно-управляемая структура, определяемая как

$\mathbf{A} = (\boldsymbol{x},\ \boldsymbol{y},\ \boldsymbol{F},\ \boldsymbol{f}_{\mathbf{cur}},\ \boldsymbol{f}_1)$, где:

1. $\boldsymbol{x}$ – входная переменная с допустимыми значениями из множества $\boldsymbol{I} = \{1, 2, 3, \ldots, \boldsymbol{m}, \ldots, \boldsymbol{M}\}$;
2. $\boldsymbol{y}$ – выходная переменная с допустимыми значениями из множества $\boldsymbol{O} = \{1, 2, 3, \ldots, \boldsymbol{n}, \ldots, \boldsymbol{N}\}$;
3. $\boldsymbol{F} = \{\boldsymbol{f}_1, \ldots, \boldsymbol{f}_k, \ldots, \boldsymbol{f}_K\}$ – множество альтернативных функций-состояний, каждая из которых содержит две подфункции: а) подфункцию выходов $\boldsymbol{fo}_k$, которая задает значение выходной переменной $\boldsymbol{y}$, и б) под-функции переходов $\boldsymbol{fs}_k$, которая задает функцию-состояние для следующего цикла из множества $\boldsymbol{F}$;
4. $\boldsymbol{f}_{\mathbf{cur}} \in \boldsymbol{F}$ – текущая функция-состояние автомата для рассматриваемого цикла;
5. $\boldsymbol{f}_1 \in \boldsymbol{F}$ – начальная функция-состояние автомата, текущая функция-состояние на начало работы автомата.

Для рассматриваемого цикла конечный автомат полностью определен его текущей функцией-состоянием $\boldsymbol{f}_{\mathbf{cur}}$.

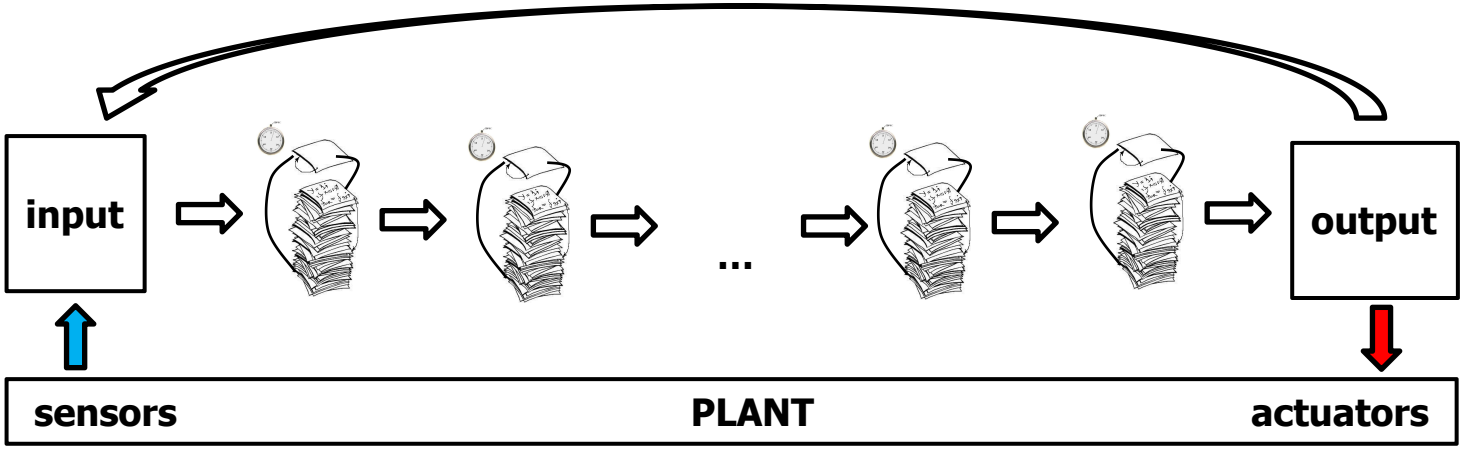
Модель процесса и гиперпроцесса

процесс = FSM

+ пассивные состояния
+ межпроцессные операции
+ time-in-state таймер
+ операции тайм-аута
+ параллелизм исполнения
+ обновленная терминология



hyper-process =
набор взаимодействующих процессов



Reflex: “Си с процессами”



Особенности языка/транслятора

- семантика основана на модели гиперпроцесса
- Си-подобный синтаксис
- классическая модель циклической активации с постоянным/переменным периодом
- специальные средства привязки к модулям связи с объектом
- расширенный синтаксис, доступ к периферии контроллера (node, import, wait, slice, ISR, function, etc.)
- портируемость через Си-проекцию программы

```
1   program Dryer {
2       clock 0t10ms;
3
4   import ATmega128Vectors
45  import ATmega128VectorsRegistersAndBits
865
866     bool dryer_control as output (read = PINB, wr
867     bool hands_under_dryer as input (read = PIND,
868
869     const bool ON = true;
870     const bool OFF = false;
871     const time TIMEOUT = 0t3s;
872
873     active process Dryer {
874         state Wait {
875             if (hands_under_dryer) {
876                 dryer_control = ON;
877                 set state Work;
878             }
879         }
880
881         state Work {
882             if (hands_under_dryer) reset timer;
883             timeout (TIMEOUT) {
884                 dryer_control = OFF;
885                 set state Wait;
886             }
887         }
888     }
889 }
```

Расширение языка IEC 61131-3 Structured Text



Особенности poST / IDE

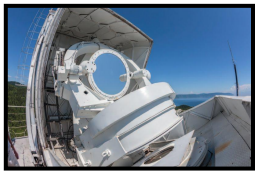
- целевая группа – IEC61131-3/ IEC61499 сообщество
- синтаксис основанный на грамматике IEC 61131-3 Structured Text (Pascal)
- концептуальная совместимость с IEC 61131-3
- синтаксически ориентированный редактор на базе Eclipse/Xtext
- портируемость: C / ST / XML-exchange

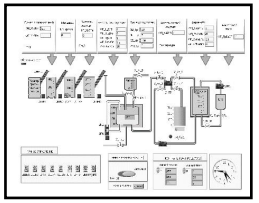
```
PROGRAM Dryer
    VAR_INPUT
        hands : BOOL;
    END_VAR
    VAR_OUTPUT
        control : BOOL;
    END_VAR
    PROCESS Dryer
        STATE Wait
            IF hands THEN
                control := TRUE;
                SET STATE Work;
            END_IF
        END_STATE
        STATE Work
            IF hands THEN
                RESET TIMER;
            END_IF
            TIMEOUT T#2s THEN
                control := FALSE;
                SET STATE Wait;
            END_TIMEOUT
        END_STATE
    END_PROCESS
END_PROGRAM
```

Практическая апробация в реальных проектах

Системы управления
Установка по выращиванию монокристаллического кремния методом Чохральского (Красноярск)
Прецизионная углоизмерительная машина (Москва)
Большой Солнечный Вакуумный телескоп (Листвянка, Байкал)
Встраиваемые системы
Установка по напылению халькогенидных пленок (Новосибирск)
Станция пробоподготовки (Новосибирск)
Промышленный коммуникационный адаптер для крупногабаритных весов (Германия)



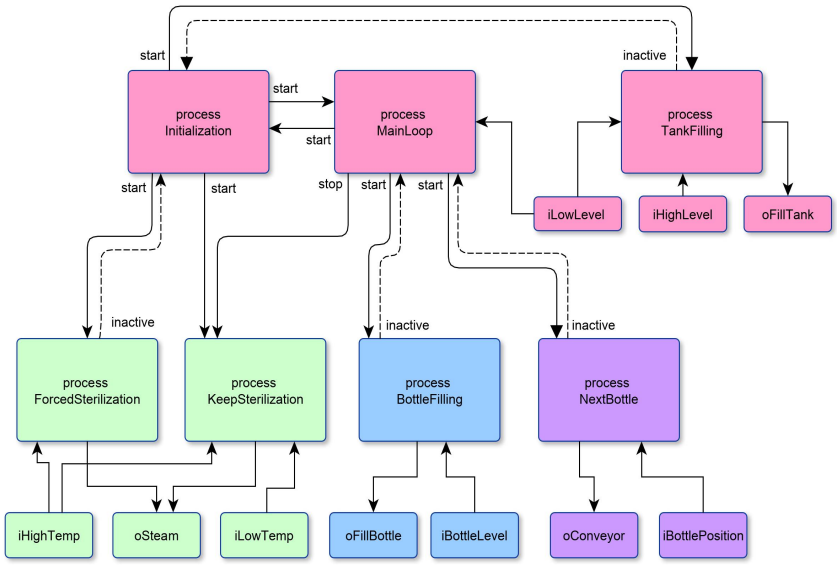





Топологически независимый метод развертывания

Предложена концепция топологически независимой спецификации алгоритмов распределенного управления на основе парадигмы процессно-ориентированного программирования:

- централизованное описание,
- разбиение на множество независимых или слабозависимых кластеров,
- развертывание полученных кластеров (или групп кластеров) на отдельных микроконтроллерных узлах распределенной системы управления.



Пример разбиения алгоритма на слабосвязанные кластеры

Предложен алгоритм выделения кластеров на основе анализа возможного недетерминизма при доступе к входным/выходным сигналам и межпроцессном взаимодействии, в том числе, в силу циклической зависимости процессов по управлению.

Экспериментальные исследования подхода на модельных задачах управления линией розлива бутылок и шлюзом показали, что процесс-ориентированная спецификация обеспечивает мелкозернистое разбиение алгоритма управления на слабосвязанные кластеры, которые в среднем содержат по два процесса.

EDTL логика для инженерии требований

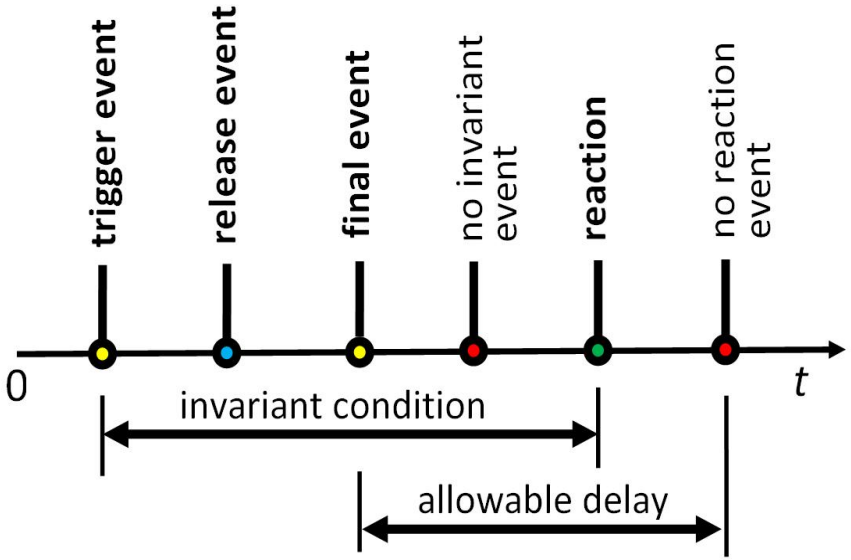
Мотивация:
Необходимость иметь нотацию для требований понятную как инженерам-технологам, так и программистам

Важность

- концептуальное несоответствие формальных методов спецификации требований с технологическим описанием алгоритмов управления
- большинство ошибок в системах управления вызваны проблемами формулировки требований, а совсем не проблемами кодирования

Решение - 6-ти компонентный паттерн:

- триггерное событие,
- событие отмены,
- финализирующее событие,
- допустимый интервал формирования реакции,
- инвариант,
- реакция.



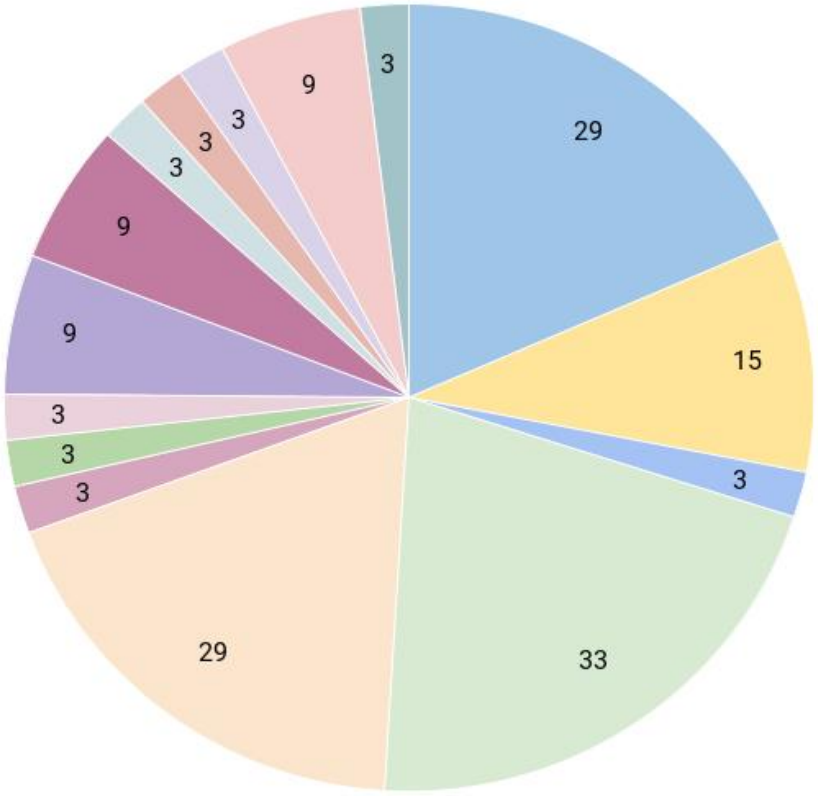
Классификация EDTL требований

Trigger event	Release	Final	Delay	Invariant	Reaction
H.**RE AND NOT** *D*	**FALSE**	**TRUE**	**TRUE**	**TRUE**	*D*
H **AND** *D*	*D*.**H** >= #T1h	**TRUE**	**TRUE**	**TRUE**	*D*
NOT *H* **AND NOT** *D*	**FALSE**	**TRUE**	**TRUE**	**TRUE**	**NOT** *D*
D **AND** *H*.**FE**	*H*	**TAU**(#T1s)	**TRUE**	*D*	**NOT** *D*
D.**RE**	**NOT** *D*	**TAU**(#T1h)	**TRUE**	**TRUE**	**NOT** *D*

Предложены правила преобразований EDTL-требований для значений TRUE/FALSE/var, всего 3^6 (729) вариантов.

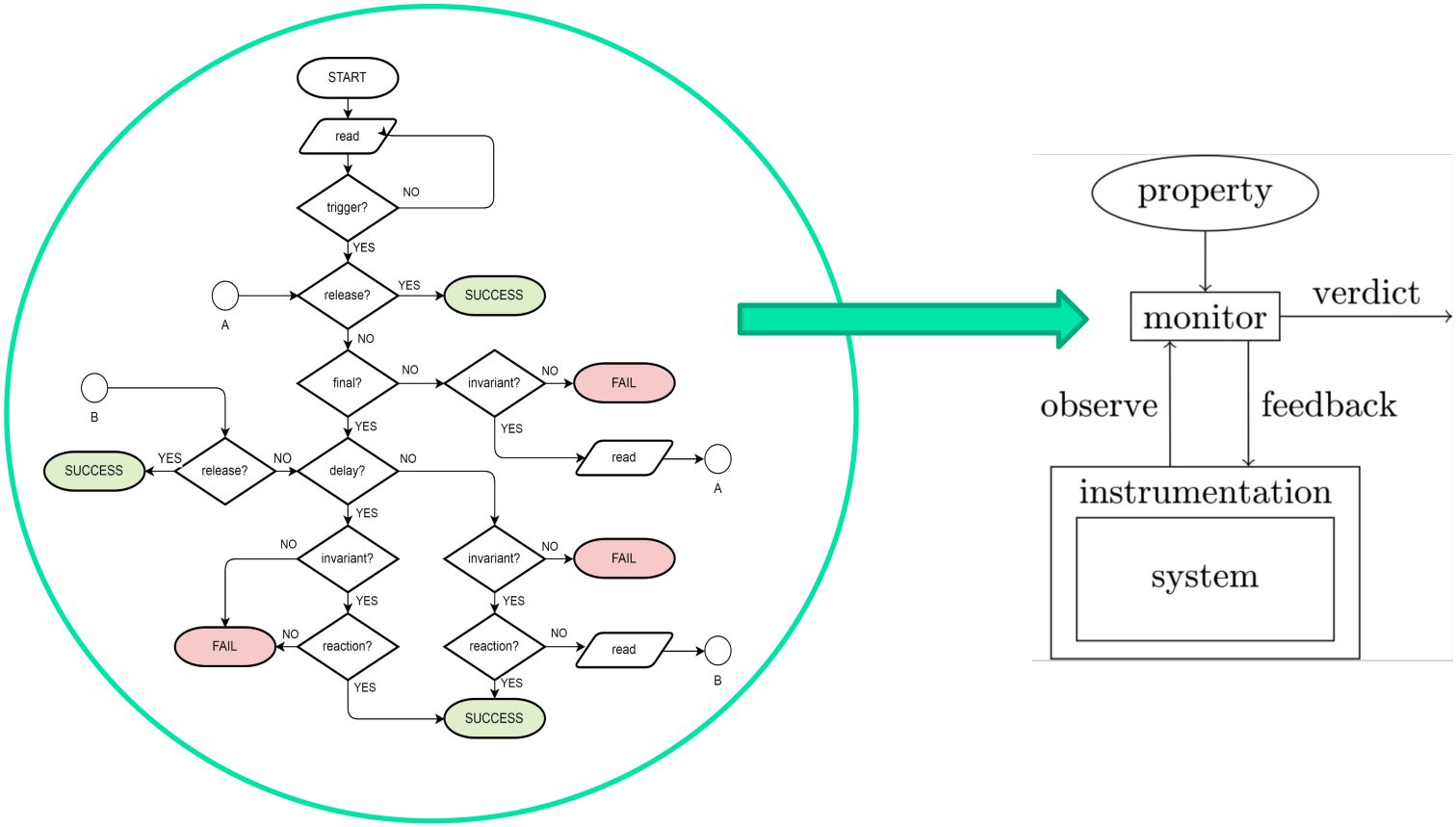
Выявлена достаточно богатая семантика:

- 2 тривиальных класса (459 TRUE/ 33 FALSE), и оставшиеся 237:
- 80 одноэлементных класса,
- 15 многоэлементных класса.



Распределение классов с нетривиальной семантикой

Автогенерация верифицирующих EDTL-автоматов



Предложен и реализован алгоритм автоматической генерации алгоритмов (Си) по EDTL-спецификации (для метода run-time верификации):

- запуск экземпляра по триггерному событию,
- контроль завершения процедуры (не завершено, нормальное завершение, завершение с ошибкой)

Концептуальная основа автоматизации

Преобразовательная (инженерная) деятельность – создание новых артефактов, изменение реальности

$$W + A \rightarrow W'$$

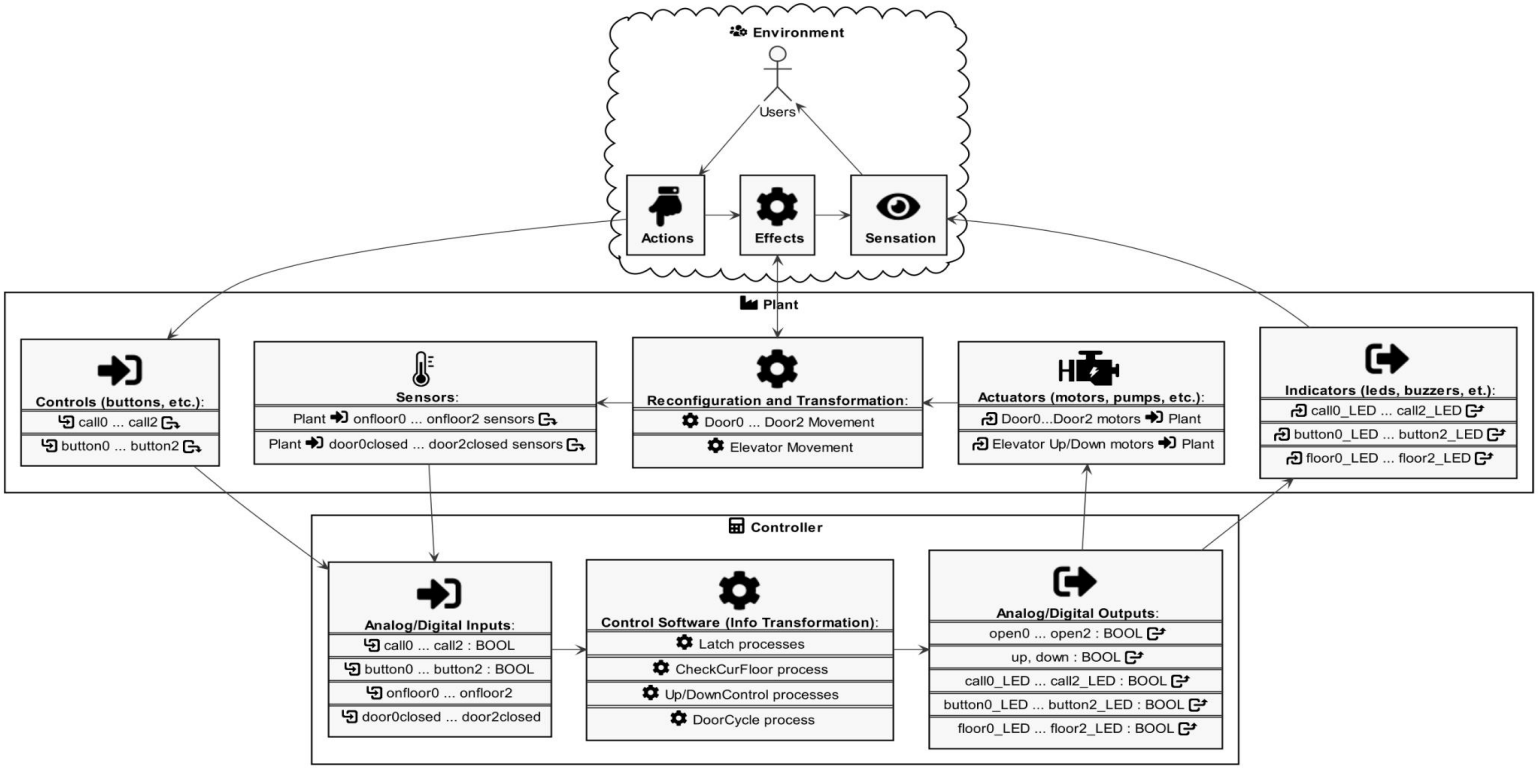




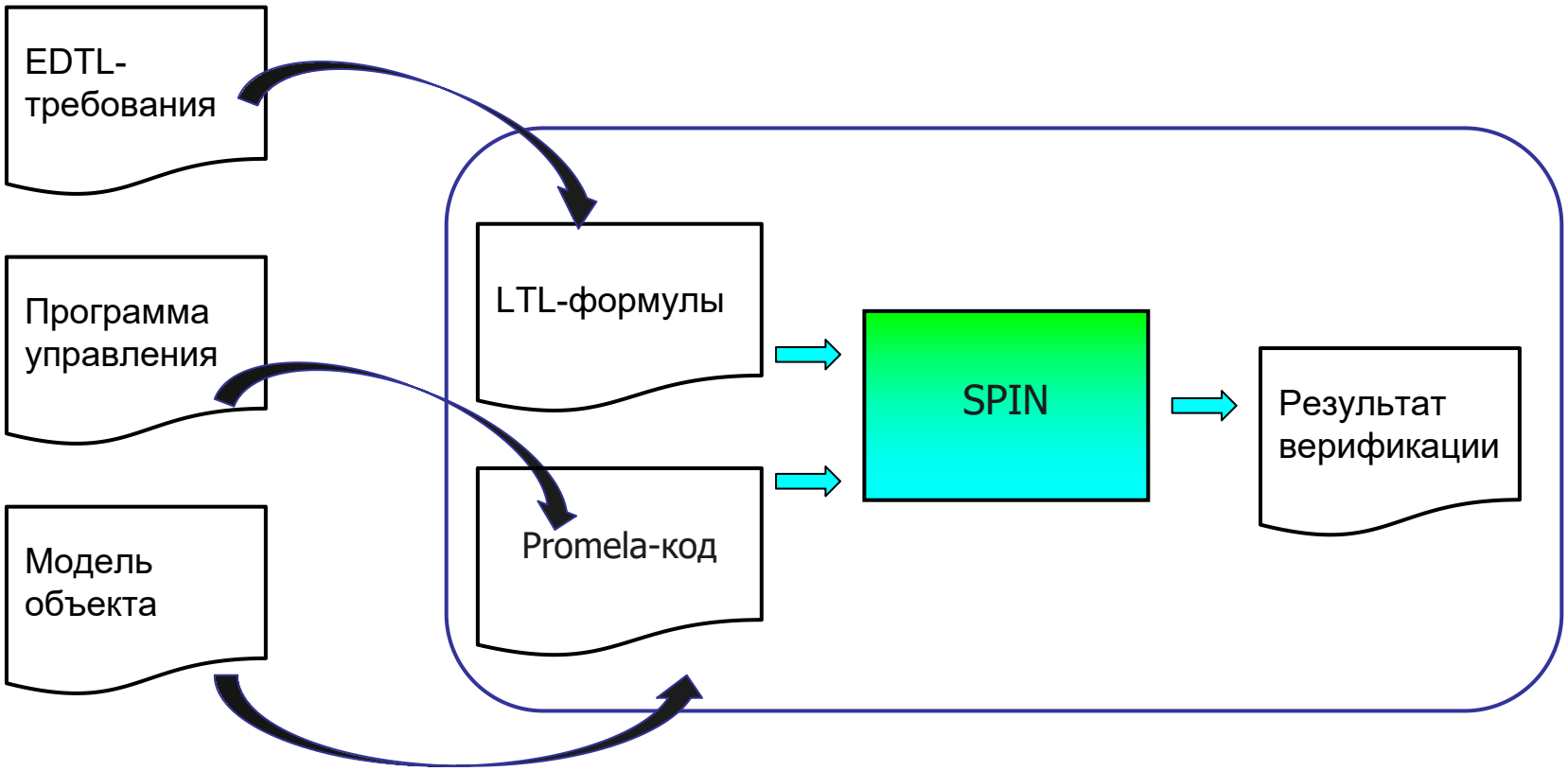
Киберфизические диаграммы системы управления

Элементы киберфизической диаграммы:

- внешняя среда (пользователь, действия пользователя, инструкции, эффекты)
- физическая часть (органы управления, датчики, поведение, актуаторы, индикаторы)
- управляющее устройство (входные/выходные сигналы, алгоритм управления)

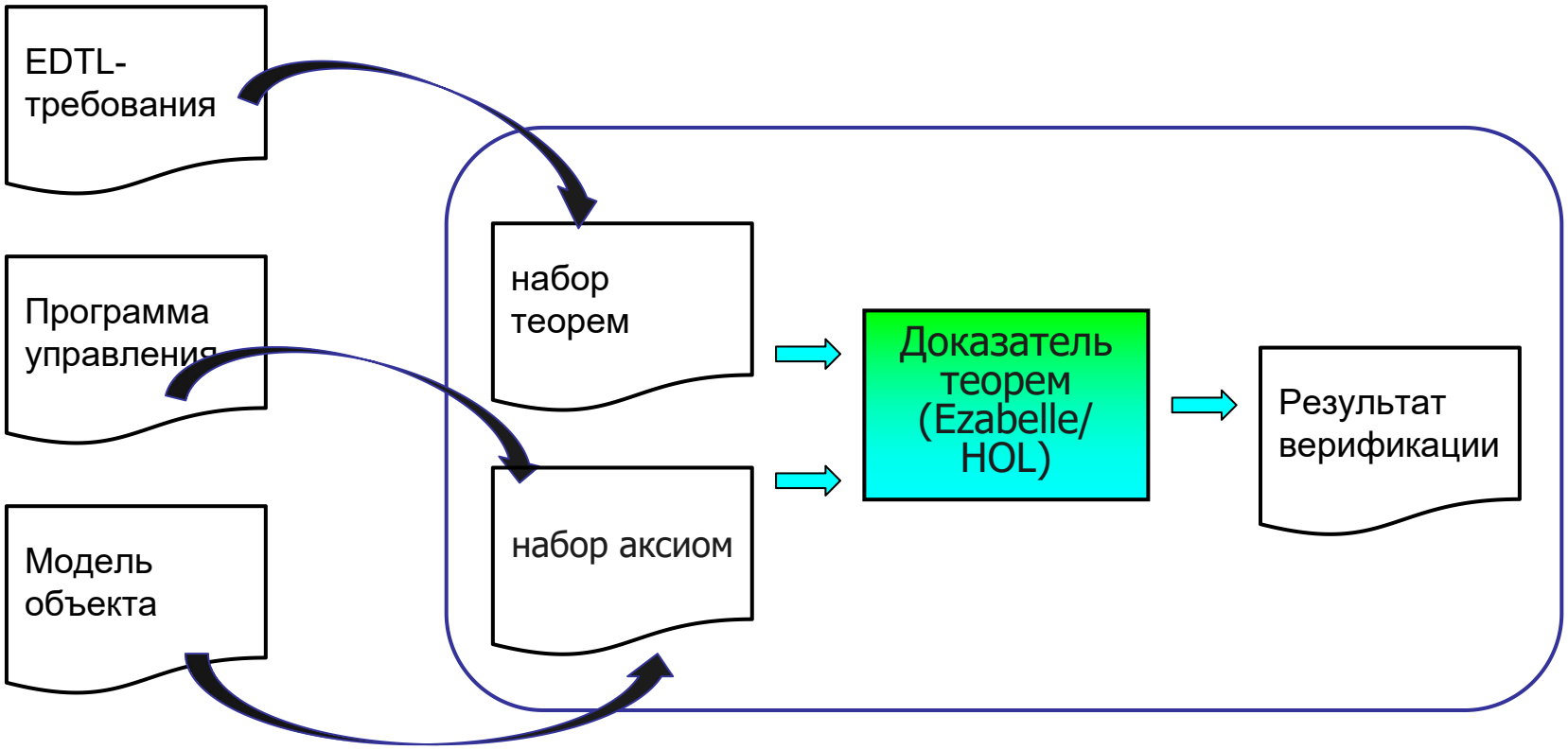


Верификация методом проверки моделей



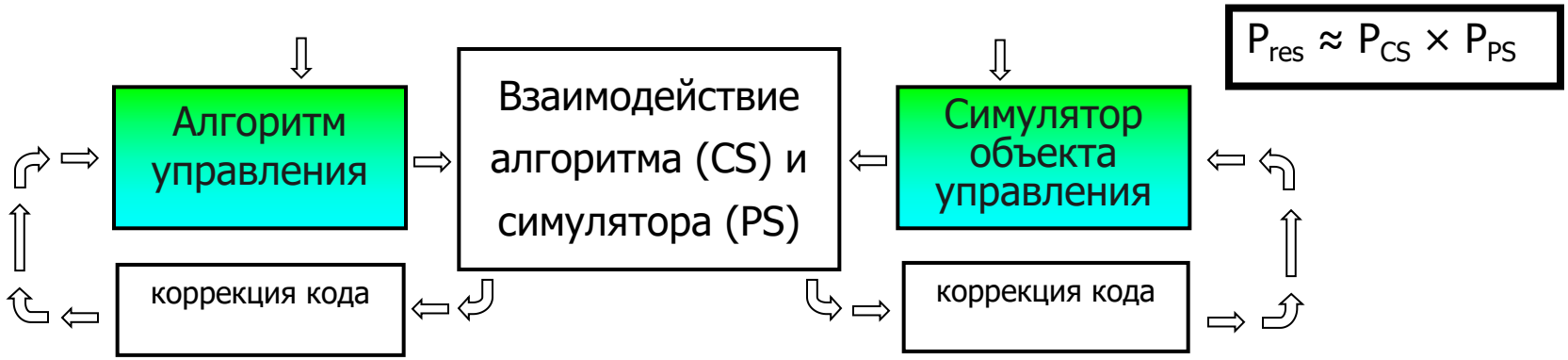
- автоматическая генерация требований в виде LTL-формул
- автоматическая генерация модели алгоритма и модели объекта на языке Promela по тексту программы
- формальная верификация в среде SPIN и генерация отчета

Дедуктивная верификация poST-программ



- автоматическая генерация условий корректности
- автоматическая генерация проекции алгоритма/модели объекта в виде набора аксиом
- шаблонизация стратегий доказательства (автоматизация доказательств)
- дедуктивная верификация, доказательство от противного
- генерация отчета

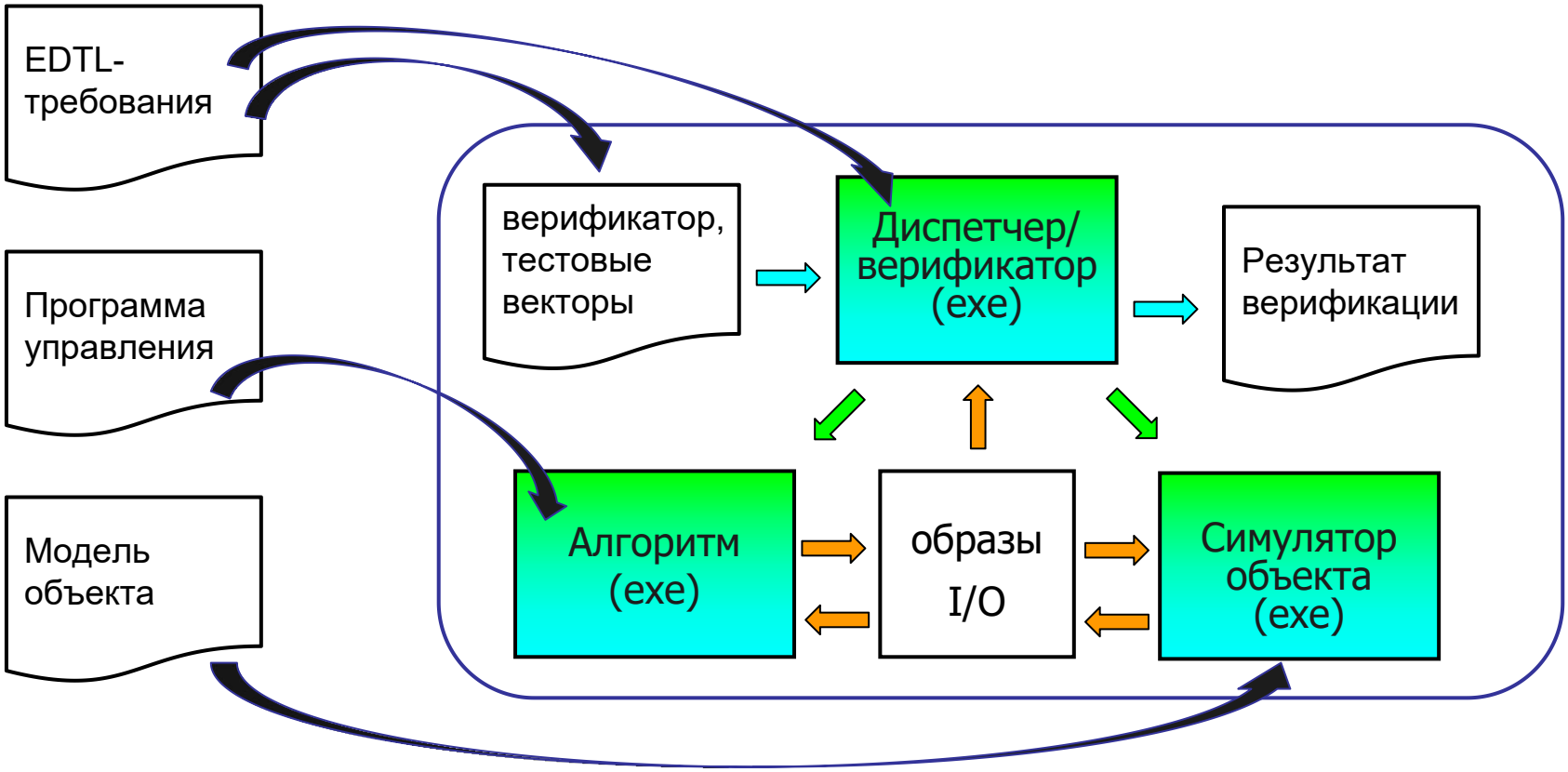
Итерационная разработка ПО на цифровых моделях



Преимущества:

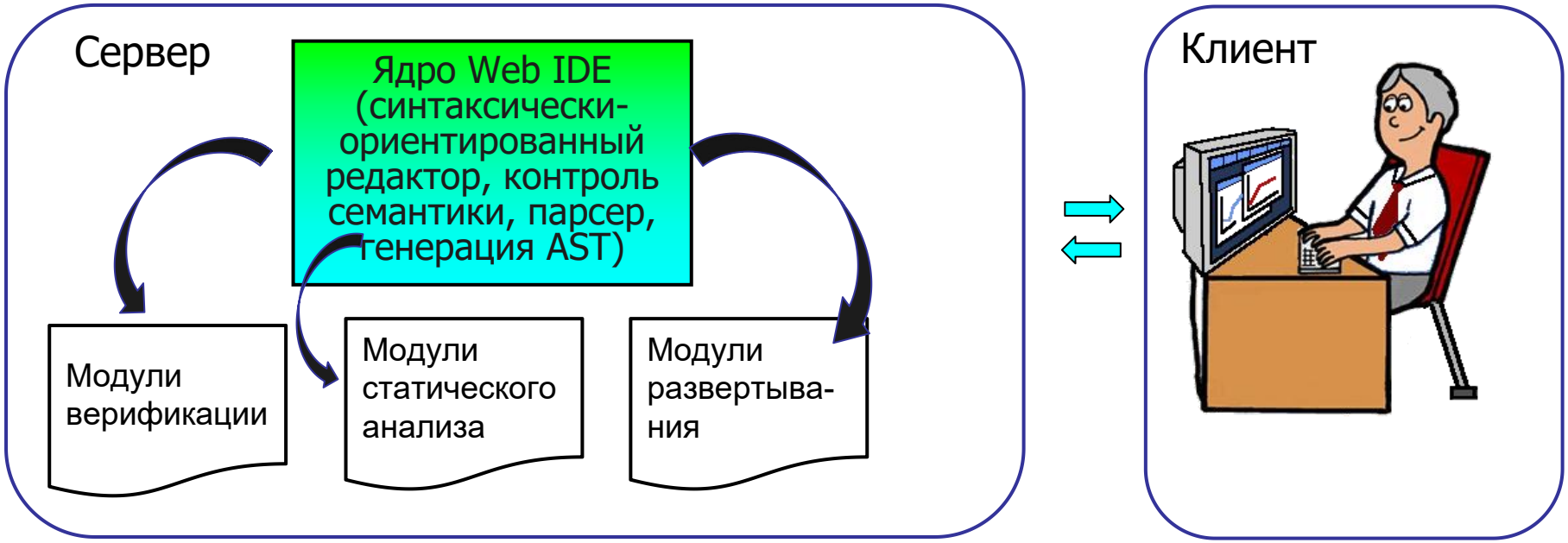
- тестирование с начала разработки, итерационные (agile) техники разработки
- возможность контроля процесса разработки, повышение управляемости проекта, снижение психологической нагрузки на разработчиков
- сокращение стоимости пусконаладки, снижение рисков получения отрицательного результата
- возможность включения в процесс разработки широкого круга специалистов, в том числе, со стороны Заказчика
- по окончанию проекта – использование в качестве цифровых двойников, создание тренажерного комплекса для подготовки операторов

Динамическая верификация КФС



- автоматическая генерация верифицирующего кода и тестовых векторов по EDTL-требования
- автоматическая генерация исполняемого кода по тексту программы
- автоматическое тестирование и генерация отчета

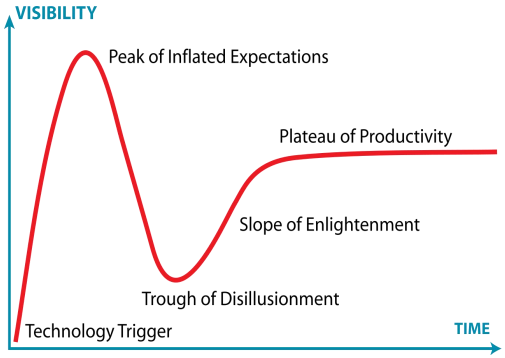
Облачные технологии разработки (WebIDE)

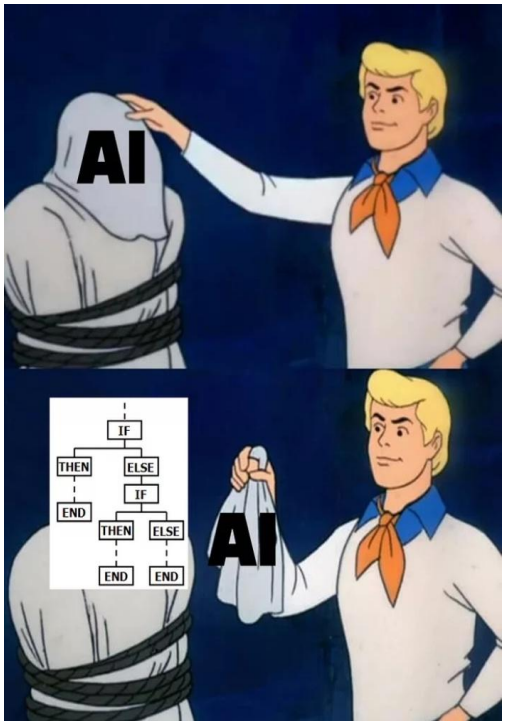


Клиент-серверная архитектура, микросервисная архитектура на базе стека открытых технологий Eclipse/Xtext, Theia, Che, Kubernetes:

- гибкость, масштабируемость
- снижение стоимости владения

Статистические методы моделирования





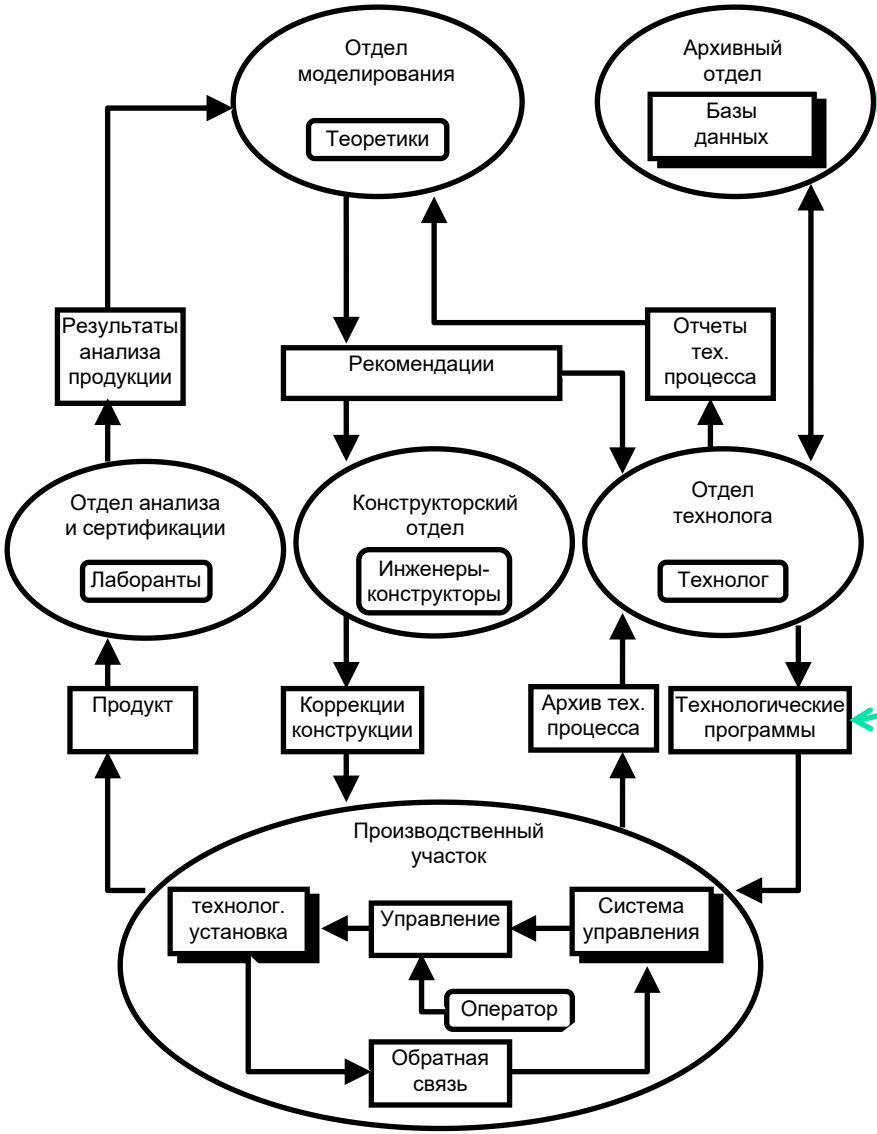
AI, **ML**, **LLM, etc**…

Опасности hype-tech

Перспективные направления исследований применимости ИИ в задачах создания систем промышленной автоматизации:

- моделирование технологических процессов
- генерация интерпретируемого кода
- генерация интерпретируемых требований
- контроль качества требований
- мутагенная верификация ПО
- поиск неустойчивости в ПО
- поиск проблемных мест в ПО

Моделирование технологических процессов



Перспективная область использования: многопараметрические, многокритериальные тех процессы с отсутствием масштабной инвариантности (химия, металлургия, биотехнологии).

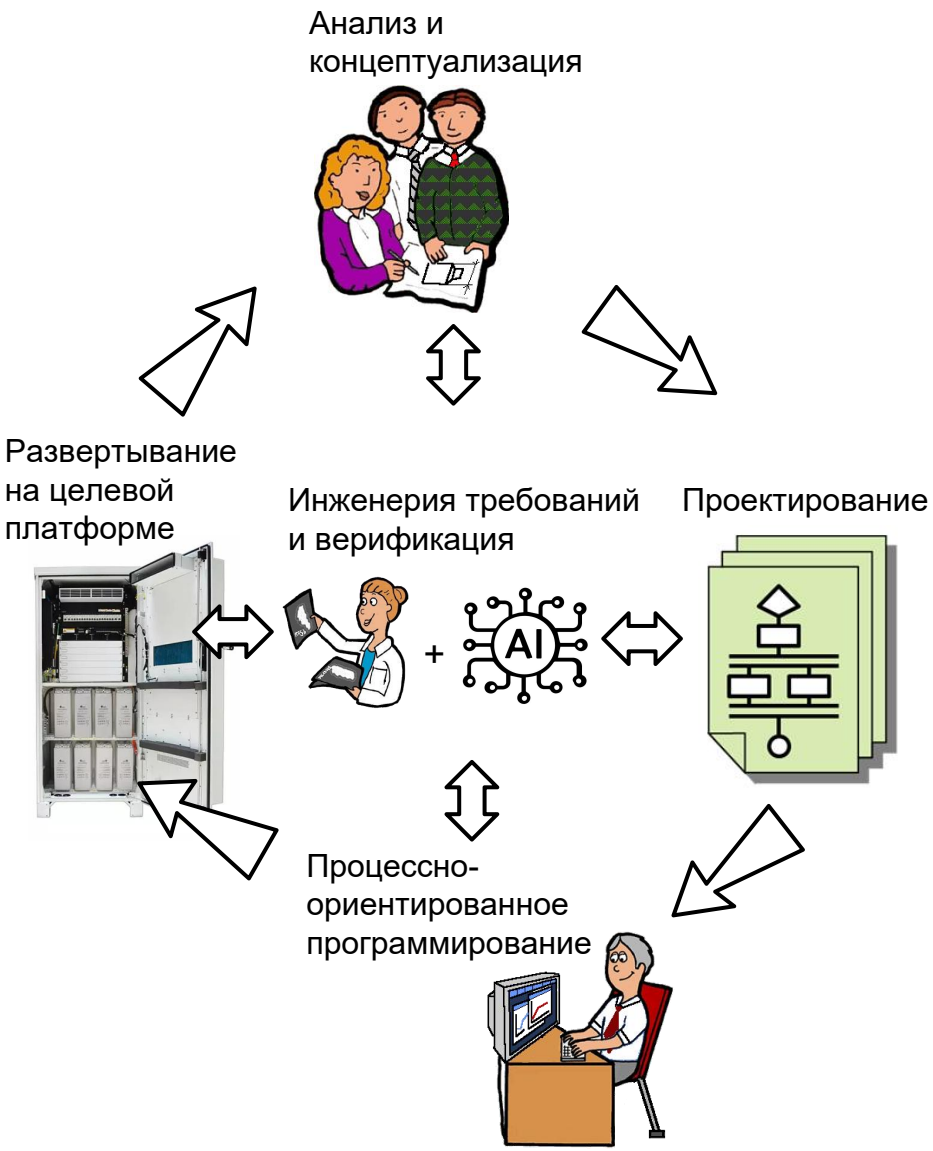
Перспективный способ использования: построение моделей по имеющимся архивам, определение режимов ведения технологического процесса по вводимым критериям эффективности

Кодирование, инженерия требований, верификация

- генерация интерпретируемого кода
- генерация интерпретируемых требований
- контроль качества требований
- мутагенная верификация ПО
- поиск неустойчивости в ПО, в том числе в AI-генерациях
- поиск проблемных мест в ПО

Проблемы	Преимущества
отсутствие достаточного количества статистики, датасетов	соответствие технологическому описанию технологического процесса
необходимость пресуппозиций, фоновых знаний, соблюдения ГОСТов, в частности, по безопасности	интерпетируемость за счет использовании проблемно-ориентированного понятийного аппарата, языка
вопрос ответственности (юридическая плоскость)	сокращение трудоемкости создания качественного ПО
недетерминированность (вероятностный характер)	поиск скрытых закономерностей, анализ кода на предмет скрытых проблем
эрозия инженерных компетенций	возможность построения цифровых моделей для сложных тех. процессов, построение генераторов режимов по заданным критериям

Предлагаемый стек высокоуровневых технологий



Набор разработанных методов и инструментальных средств (модели, языки спецификации алгоритмов и требований, методы верификации, IDE) обеспечивает принципиальное преимущество по сравнению с практикуемыми подходами, а именно:

1. Высокое качество создаваемых программ
2. Сокращение стоимости и времени разработки
3. Высокая сопровождаемость
4. Снижение квалификационных требований к разработчикам
5. Высокая управляемость проектов
6. Независимость от зарубежных контрагентов
7. Перспективы усиления за счет интеграции с ИИ без потери качества и уровня контроля

Институт автоматики и электрометрии СО РАН



Контакты:
Лаборатория киберфизических систем ИАиЭ СО РАН
д-р техн. наук Владимир Евгеньевич Зюбин
Email: zyubin@iae.nsk.su